PYTHON FOR BEGINNERS · SESSION 5

Loops

Repeat work with for and while, count with range(), and control a loop with break and continue.

Recap

Ask the class:

- What is the difference between `if` and `elif`?
- What is the output of `print(10 > 5 and 3 > 8)`?
- Why does indentation matter in Python?

Why do we need loops?

A **loop** repeats a block of code. Without a loop, repeating a task means **copying the same line again and again**.

Without a loop

```
print("Hello")
print("Hello")
print("Hello")
print("Hello")
print("Hello")
```

OUTPUT

```
Hello
Hello
Hello
Hello
Hello
```

With a loop: same result, and it still works for 5000 times

```
for i in range(5):
    print("Hello")
```

OUTPUT

```
Hello
Hello
Hello
Hello
Hello
```

Two kinds of loop

LOOP	USE IT WHEN
`for`	you know **how many times**, or you want to go through **every item**
`while`	you want to repeat **until a condition becomes False**

The for loop

Syntax

```
for variable in sequence:
    statement
```

The loop **takes one item at a time** from the sequence, stores it in the variable, and runs the indented block. When the items run out, the loop ends.

Looping through a string

```
for letter in "Python":
    print(letter)
```

OUTPUT

```
P
y
t
h
o
n
```

Looping through a list (preview)

A **list** holds several values in square brackets. You will study lists later; for now, see that a loop can go through one.

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print("I like", fruit)
```

OUTPUT

```
I like apple
I like banana
I like mango
```

Ask

What is the loop variable in `for letter in "Python":`? What value does it hold in the third round?

Printing on one line

`print()` normally ends with a new line. `end=" "` changes that.

```
for i in range(1, 6):
    print(i, end=" ")
```

OUTPUT

```
1 2 3 4 5
```

The range() function

`range()` makes a **sequence of numbers** for a loop. It has three forms: `range(stop)`, `range(start, stop)` and `range(start, stop, step)`.

CALL	NUMBERS IT GIVES
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8
`range(10, 0, -2)`	10, 8, 6, 4, 2

- `start` is **0** if you leave it out.
- `step` is **1** if you leave it out.
- The **stop number is never included**, just like in slicing.
- A **negative step** counts down.

See the numbers

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 10, 2)))
print(list(range(10, 0, -2)))
```

OUTPUT

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 2, 4, 6, 8]
[10, 8, 6, 4, 2]
```

Sum of 1 to 10

```
total = 0
for n in range(1, 11):
    total += n
print(total)
```

OUTPUT

```
55
```

A multiplication table

```
number = 5
for i in range(1, 11):
    print(number, "x", i, "=", number * i)
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Ask

How would you print the numbers from 10 down to 1?

The while loop

Syntax

```
while condition:
    statement
```

Python **checks the condition before every round**. While it is True, the block runs again. When it becomes False, the loop ends.

Counting with while

```
count = 1
while count <= 5:
    print(count)
    count += 1
print("Done")
```

OUTPUT

```
1
2
3
4
5
Done
```

- **Start** the counter before the loop (`count = 1`).
- **Test** it in the condition (`count <= 5`).
- **Change** it inside the loop (`count += 1`). Without this the loop never ends.

The loop may run zero times

If the condition is False at the very start, the block **never runs**.

```
count = 10
while count <= 5:
    print("This never prints")
print("Finished")
```

OUTPUT

```
Finished
```

Sum of the digits of a number

```
n = 1234
total = 0
while n > 0:
    total += n % 10
    n //= 10
print(total)
```

OUTPUT

```
10
```

Ask

In the digits program, what are the values of `n` after each round?

Infinite loops

A loop whose condition **never becomes False** runs forever. This is called an **infinite loop**.

Do NOT run this: it never stops

```
count = 1
while count <= 5:
    print(count)
    # count never changes, so count <= 5 is always True
```

If your program gets stuck

Press **Ctrl + C** in the terminal to stop it. In the browser boxes on this site, close and reopen the tab.

Waiting for the right answer

A `while` loop is perfect when you do not know how many tries the user needs.

Run this one in VS Code or a terminal

```
password = ""
while password != "python123":
    password = input("Enter password: ")
print("Access granted")
```

for or while?

SITUATION	BETTER CHOICE
Repeat exactly 10 times	`for`
Go through every letter of a word	`for`
Keep asking until the answer is correct	`while`
Repeat until a number reaches 0	`while`

The break statement

`break` **stops the loop immediately** and jumps to the first line after it.

```
for number in range(1, 20):
    if number % 7 == 0:
        print("First multiple of 7:", number)
        break
print("Loop ended")
```

OUTPUT

```
First multiple of 7: 7
Loop ended
```

while True with break

`while True` repeats **forever**, so a `break` inside decides when to stop. This is a common pattern.

```
count = 0
while True:
    count += 1
    if count == 3:
        break
print(count)
```

OUTPUT

```
3
```

Ask

What would the first program print without the `break` line?

The continue statement

`continue` **skips the rest of the current round** and goes straight to the next one. The loop does **not** stop.

Skip the even numbers

```
for i in range(1, 8):
    if i % 2 == 0:
        continue
    print(i)
```

OUTPUT

```
1
3
5
7
```

break and continue together

STATEMENT	WHAT IT DOES
`break`	ends the **whole loop**
`continue`	ends only the **current round**

```
for i in range(1, 10):
    if i == 3:
        continue
    if i == 6:
        break
    print(i)
```

OUTPUT

```
1
2
4
5
```

Loops inside loops

A loop can sit **inside another loop**. For every round of the outer loop, the inner loop runs **completely**.

A star triangle

```
for i in range(1, 5):
    print("*" * i)
```

OUTPUT

```
*
**
***
****
```

A multiplication grid

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

OUTPUT

```
1 2 3
2 4 6
3 6 9
```

- The inner loop is indented **one more level**.
- `print()` with nothing inside **ends the line** after each row.

Common mistakes

MISTAKE	WHAT HAPPENS	FIX
Forgetting the colon	SyntaxError	end the `for` or `while` line with `:`
`range(1, 5)` expecting 5	stops at 4	use `range(1, 6)`
Not changing the counter in `while`	infinite loop	update it inside the loop
Wrong indentation	a line runs once, or every round	indent only what must repeat

Off by one

Try it: change it so that 5 is printed too

```
for i in range(1, 5):
    print(i)
```

OUTPUT

```
1
2
3
4
```

Inside or outside?

```
for i in range(3):
    print("inside", i)
print("outside")
```

OUTPUT

```
inside 0
inside 1
inside 2
outside
```

Practice questions

Basic questions

1. Print the numbers from 1 to 10.
2. Print the even numbers from 1 to 50.
3. Print the sum of the numbers from 1 to 100.
4. Take a number and print its multiplication table up to 10.
5. Take a number and print its factorial.

Harder questions

6. Take a word and count how many vowels it has.
7. Print a word in reverse using a loop (do not use slicing).
8. Print a star triangle with 5 rows.
9. Take a number and find the sum of its digits using a `while` loop.
10. Keep asking for a password until the user types the right one, then print "Welcome".

Find the output

Work out the output first, then press Run to check

```
for i in range(1, 6):
    if i == 3:
        continue
    if i == 5:
        break
    print(i)
```

Try the sum question

Your turn: add the numbers 1 to 100, then press Run

```
total = 0
# add the numbers 1 to 100 using a for loop
print(total)
```

Homework and wrap-up

- Print a **multiplication table** in this format: `5 x 3 = 15`, using an f-string.
- Write a **number guessing game**: keep a secret number, keep asking for guesses, print "too high" or "too low", and stop with `break` when the guess is right.
- Write **FizzBuzz**: for the numbers 1 to 50, print "Fizz" for multiples of 3, "Buzz" for multiples of 5, "FizzBuzz" for multiples of both, and the number otherwise.
- Take a number and **count its digits** using a `while` loop.
- Revise the **difference between break and continue**.

Next class

Lists.